Данила Аринов

Junior-разработчик — Node.js

+7 901 431-82-98 · teivrim@gmail.com · Telegram @Smishnyavko · github.com/TeivrimOriginal · Портфолио · LinkedIn

О СЕБЕ

Делал мессенджер с реалтайм-обменом на WebSocket и серверной стороной на Node.js. Потом сознательно переписал похожую задачу на Rust, чтобы сравнить подходы к структуре проекта.

ОПЫТ РАБОТЫ

Фриланс — заказы на Kwork

`Python · по заказам на Kwork`

- Telegram-боты, парсеры сайтов, REST API, интеграции внешних API, автоматизация по ТЗ заказчиков; kwork.ru/user/teivrim.

ПРОЕКТЫ

Социальный мессенджер — Node.js

`JavaScript · Node.js · WebSocket`

- Реалтайм-обмен сообщениями на WebSocket, хранение данных на стороне сервера.
- Позже переписал похожую задачу на Rust, чтобы сравнить подходы.

anime-sync-api — REST API каталога аниме

`FastAPI · PostgreSQL 16 · Alembic · GIN`

- Асинхронный стек: FastAPI, SQLAlchemy 2.0 (`asyncpg`), Pydantic v2, httpx. Схемы запросов и ответов разделены.
- Три таблицы с внешними ключами и CHECK-ограничениями; `UNIQUE (source, external_id)` — внешний id уникален в пределах источника, потому что у AniList и Jikan нумерация разная.
- Полнотекстовый поиск на `tsvector` с индексом GIN по названию, оригинальному названию и жанрам. Выражение индекса и выражение запроса вынесены в одну константу, чтобы они не разошлись.

TaskHub API — сервис управления задачами

`FastAPI · PostgreSQL 17.4 · Celery · Redis`

- Асинхронный сервис на FastAPI и SQLAlchemy 2.0: `asyncpg`, Pydantic v2, разделение схем запросов и ответов, OpenAPI 3.1 из коробки.
- Аутентификация JWT с ротацией refresh-токенов и RBAC: роли проверяются на уровне зависимости, доступ к эндпоинту нельзя получить без проверки прав.
- Фоновые задачи на Celery, Redis как брокер и кэш; задачи выполняются синхронно при `CELERY_TASK_ALWAYS_EAGER`, поэтому тесты не зависят от воркера.

НАВЫКИ

JavaScript: Node.js, WebSocket, DOM, асинхронный JS

HTTP и API: REST, коды ответов, идемпотентность, JWT с ротацией refresh, RBAC, CORS, фильтрация, пагинация, OpenAPI 3.1, WebSocket

SQL и PostgreSQL: PostgreSQL 16/17 (tsvector, GIN, pg_trgm, CHECK, FK CASCADE, EXPLAIN, индексы), Alembic-миграции, SQLite, JOIN/GROUP BY, запросы к регистрам

Тесты: pytest, pytest-asyncio, httpx ASGITransport, фикстуры, моки внешних API, Allure, Selenium 4, покрытие с ветвлениями

ДОПОЛНИТЕЛЬНО

Понимаю, что Node.js и Python решают одну задачу разными средствами: у меня один и тот же REST API есть на обоих стеках. Английский — чтение технической документации. Открытый код: 12 активных публичных репозиториев, у каждого проекта тесты и CI.